IT114 Module 4 Sockets Challenge

Student submission

Assignment revision: 1 | Maximum points: 10 | Student: ja692 | Submitted: 2026-10-17T22:14:39.039Z | Submission revision: 191 | Receipt: v4r_01M55YWV5ZDD21QNXVBMZW23EV | Exported: 2026-10-17T22:14:43.527Z

Worksheet link

https://courses.ethereallab.app/assignment/v4/IT114-001-F2026/it114-module-4-sockets-challenge/ja692

Grading link

https://courses.ethereallab.app/assignment/v4/IT114-001-F2026/it114-module-4-sockets-challenge/grading/ja692

AI disclosure: Completed; 1 entry; 1 declared AI use; 0 declared no AI use; policy v1

Coin Flip

Add `/flip`. The client sends the command; its handler passes the request to server logic, and the server broadcasts a result in the format `<who> flipped a coin and got <result>`.

Coin flip source

URL prompt

Provide direct GitHub file URLs for `Client.java`, `ServerThread.java`, and `Server.java` on `M4-Homework`; add multiple URLs so each file can be opened directly.

Requirements and potential deductions

- The linked source includes UCID, date, and an updated high-level solution comment. (up to -0.2 points)
- The client recognizes `/flip` and sends a request through its existing connection. (up to -0.4 points)
- The handler recognizes the request and passes it to the server's application logic. (up to -0.4 points)
- The server chooses either coin result and broadcasts `<who> flipped a coin and got <result>` as a server message. (up to -0.5 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/Client.java
https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/ServerThread.java
https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/Server.java

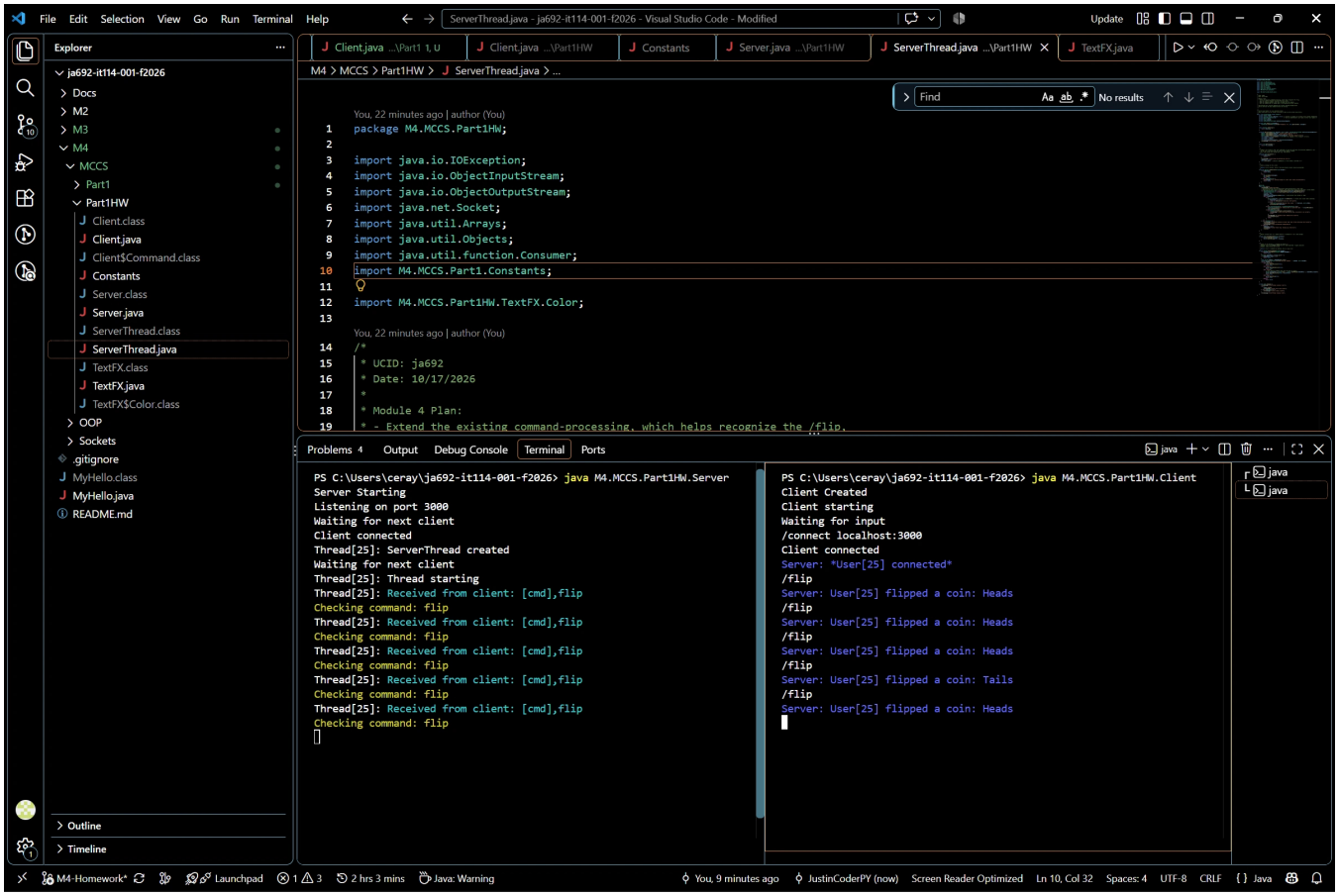
Coin flip output

Image evidence prompt

Upload readable terminal captures of five `/flip` runs with the server and at least two clients visible; use multiple images if needed.

Requirements and potential deductions

- Five command results are visible across the captures. (up to -0.5 points)
- Both clients receive the server's coin-flip result for the demonstrated runs. (up to -0.5 points)



Coin flip explanation

Response prompt

Explain how the request moves from the client through its handler to the server and how the result reaches clients.

Requirements and potential deductions

- The response explains the actual routing and result decision, rather than repeating the prompt. (up to -0.5 points)

Submitted response

When the user enters /flip, Client.java sees that command and sends a [cmd], then after, goes to a flip request through the exis socket connection. ServerThread.java later receives that request and passes it to the server using handleFlip(). Lastly, Server.j randomly chooses heads or tails, creates a message showing which user flipped the coin and the result, and sends a message server.

Private Message

Add `/pm <target id> <message>`. The server sends `PM from <who>: <message>` only to the sender and selected recipient.

Private message source

URL prompt

Provide direct GitHub file URLs for `Client.java`, `ServerThread.java`, and `Server.java` on `M4-Homework`.

Requirements and potential deductions

- The linked source includes UCID, date, and an updated high-level solution comment. (up to -0.2 points)
- The client accepts a target ID and message and sends them through the existing connection. (up to -0.4 points)
- The handler passes the private-message request to server logic. (up to -0.4 points)

- The server sends `PM from <who>: <message>` to the sender and target only. (up to -0.5 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/Client.java
https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/ServerThread.java
https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/Server.java

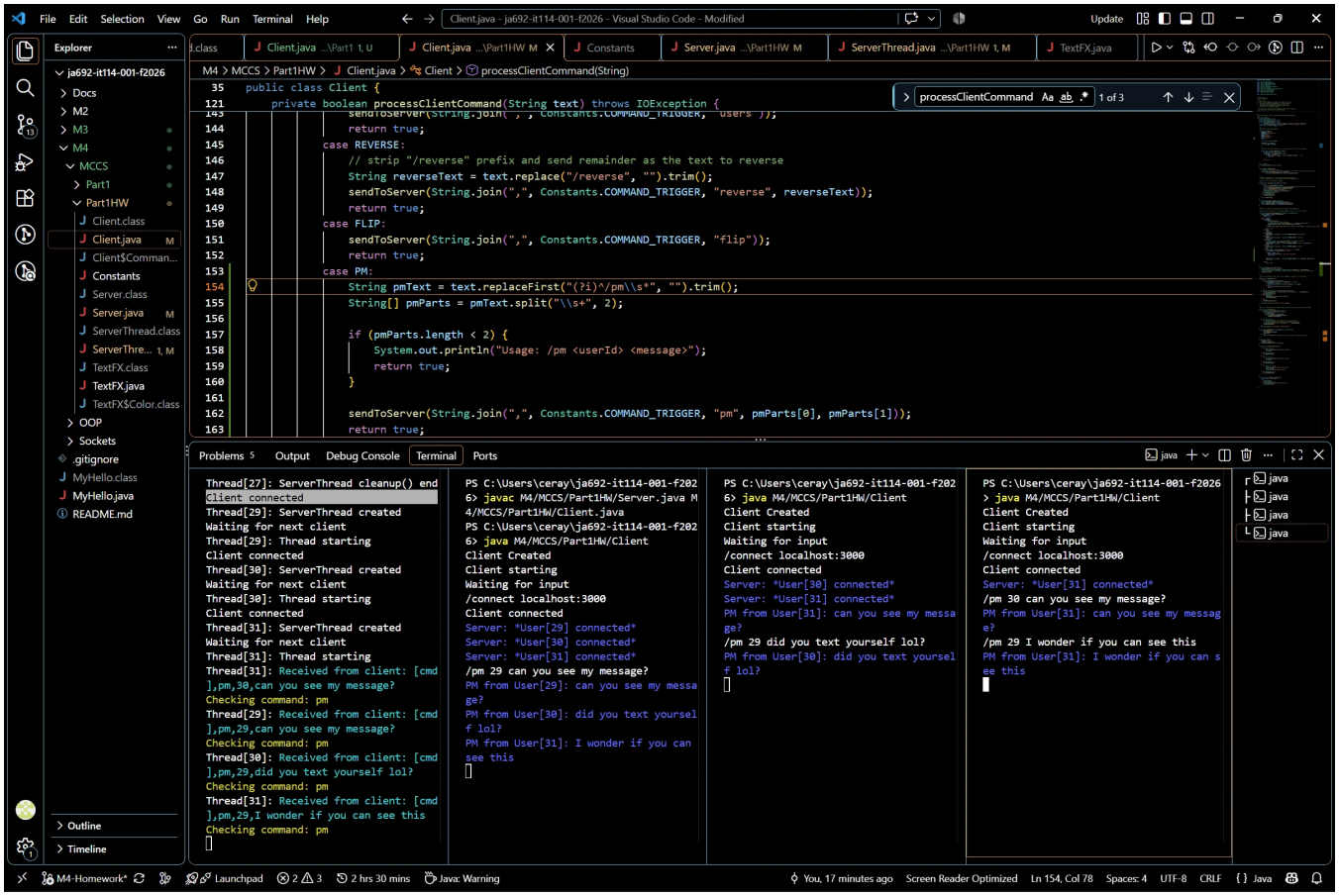
Private message output

Image evidence prompt

Upload readable captures of three `/pm` variations with the server and at least three clients visible.

Requirements and potential deductions

- Three variations with different senders or targets are visible. (up to -0.5 points)
- Only the sender and target receive each private message; the other connected client does not. (up to -0.5 points)



Private message explanation

Response prompt

Explain how your server selects the two recipients and handles a target ID.

Requirements and potential deductions

- The response explains the actual recipient selection and routing decisions. (up to -0.5 points)

Submitted response

The server uses the target ID private message command to look up the correct client number in the connectedClients. From th the target exists, the server creates the private-message text and sends it only to the sender and the target using sendToClien does not use the usual broadcast methods from the other cases, so the other connected clients do not receive the message. If target ID is not found, they will promptly tell the other user that number doesn't exist.

Shuffle Message

Add `/shuffle <message>`. The handler passes the request to server logic, which broadcasts `Shuffled from <who>: <shuffled_message>`.

Shuffle message source

URL prompt

Provide direct GitHub file URLs for `Client.java`, `ServerThread.java`, and `Server.java` on `M4-Homework`.

Requirements and potential deductions

- The linked source includes UCID, date, and an updated high-level solution comment. (up to -0.2 points)
- The client accepts `/shuffle` with message text and sends the request. (up to -0.4 points)
- The handler passes the request to server logic. (up to -0.4 points)
- The server shuffles the message and broadcasts `Shuffled from <who>: <shuffled_message>`. (up to -0.5 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/Server.java
https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/ServerThread.java
https://github.com/JustinCoderPY/ja692-it114-001-f2026/blob/M4-Homework/M4/MCCS/Part1HW/Client.java

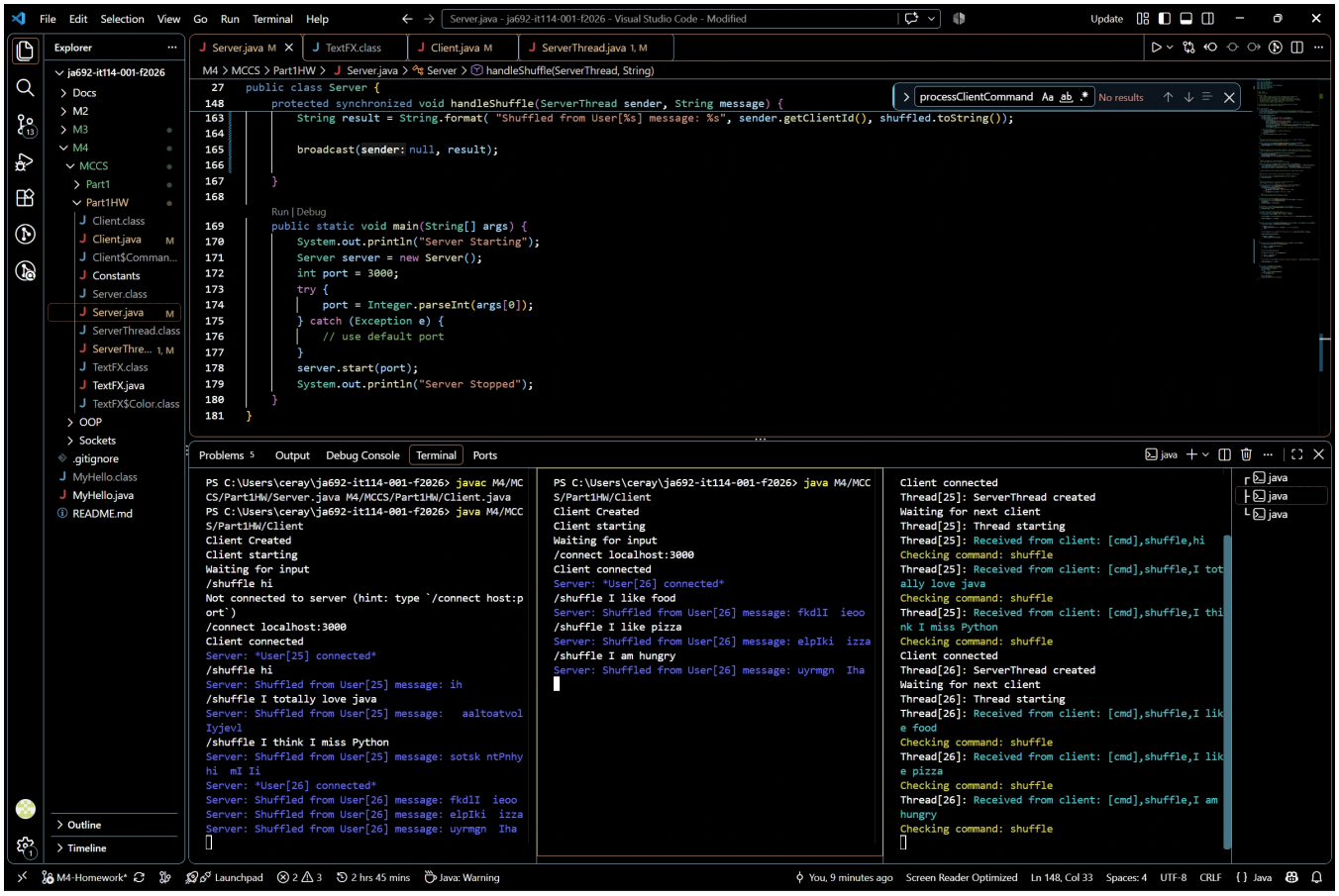
Shuffle message output

Image evidence prompt

Upload readable captures of three `/shuffle` runs with the server and at least two clients visible.

Requirements and potential deductions

- Three command results are visible across the captures. (up to -0.5 points)
- Both clients receive each demonstrated shuffled message. (up to -0.5 points)



Shuffle message explanation

Response prompt

Explain how your server shuffles the supplied message and routes its result.

Requirements and potential deductions

- The response explains the actual shuffle and broadcast decisions. (up to -0.5 points)

Submitted response

When a client sends /shuffle, the server takes that message, converting the test into individual chaters, and then stores them ir
After that, it uses Collections.shuffle() to randomize those orders of characters, building them to a new string. Once done, the s
formats the senders with the connected user ID so that every client gets the same shuffled result.

Reflection

Pull request URL

URL prompt

Submit the pull-request URL from `M4-Homework` into `main`.

Requirements and potential deductions

- The pull request opens and contains baseline, plan, and solution commits for this assignment. (up to -0.2 points)

https://github.com/JustinCoderPY/ja692-it114-001-f2026/pull/8

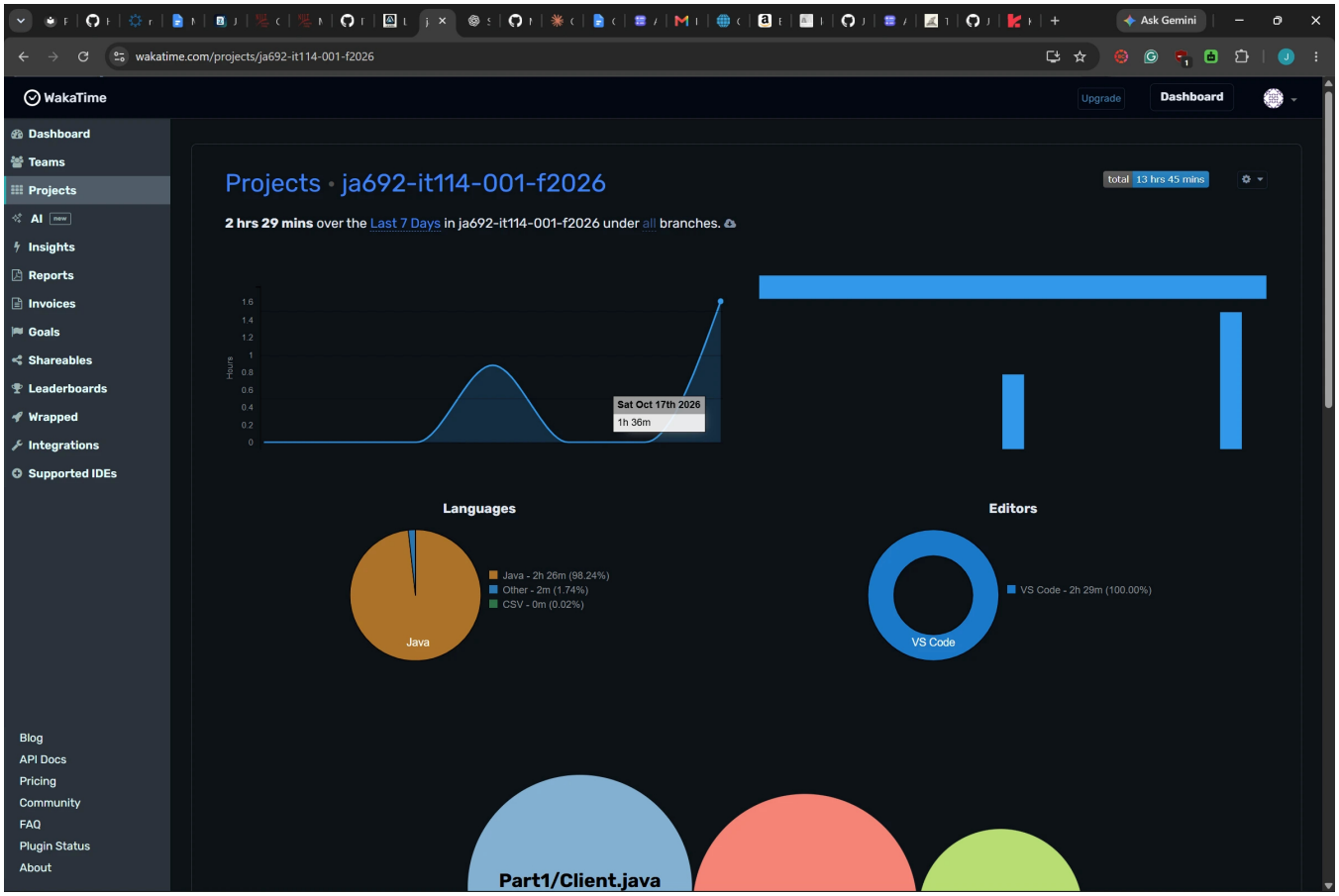
WakaTime project overview

Image evidence prompt

Upload a readable WakaTime project overview showing your repository name; time duration is not graded.

Requirements and potential deductions

- The capture identifies the assignment repository. (up to -0.1 points)



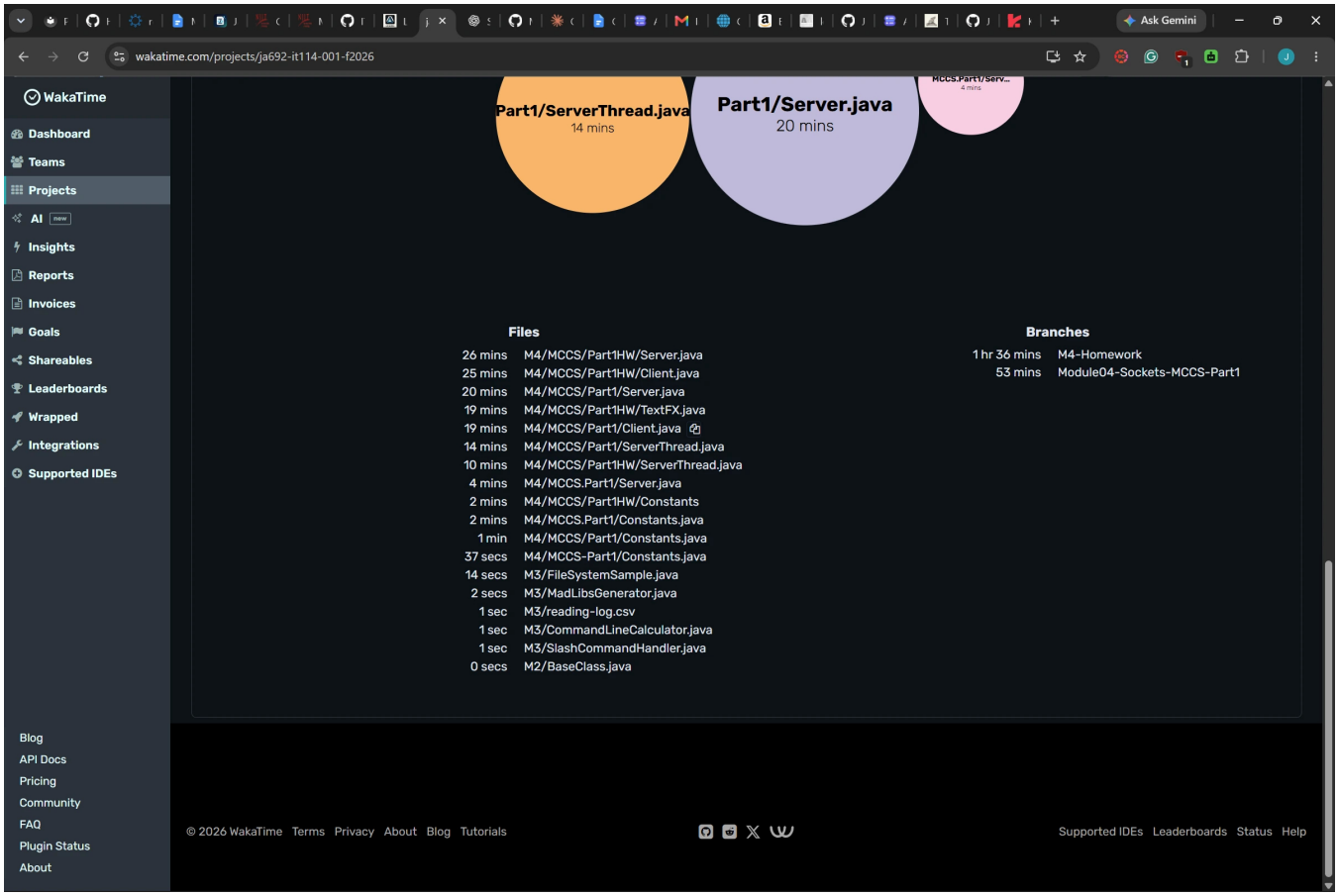
WakaTime file activity

Image evidence prompt

Upload a readable WakaTime file-activity capture for this assignment; time duration is not graded.

Requirements and potential deductions

- The capture identifies relevant Java file activity. (up to -0.1 points)



Learned or verified

Response prompt

What did you learn or verify about client/server routing or cleanup?

Requirements and potential deductions

- The response identifies a concrete lesson or verification result. (up to -0.2 points)

Submitted response

When it comes to clients and servers, I learned that those need to be constantly communicating with each other and need clea routing. On top of that, the client needs to send the correct request, which means the handler also needs to recognize the com and the server to execute or respond to the users.

Easiest part

Response prompt

Which part of this assignment was easiest, and why?

Requirements and potential deductions

- The response names a specific part and explains why it was easier. (up to -0.2 points)

Submitted response

The easiest part of the assignment has to be the /flip command because the logic was more straightforward. The server really needed to randomly choose between two results, heads or tails, and show the results to the client.

Difficult or uncertain part

Response prompt

Which part was most difficult or remains uncertain, and what did you try or check?

Requirements and potential deductions

- The response names a specific difficulty or uncertainty and a relevant check. (up to -0.2 points)

Submitted response

The hardest part for me was the shuffle command because the message had to be broken down into each of their characters b
the sentence could be randomized. I had to make sure the characters were shuffled as well in a sense where it wasn't just wor
moving around but the letters and rebuilding back to a string.